week2_notebook_student

October 27, 2026

1 HS4002 Week 2

1.1 Setup

Make sure this notebook and `gss2022_mini.csv` are in the same folder. Install packages below if needed.

[39]:
```
# !pip install pandas numpy plotnine

import pandas as pd
import numpy as np
import plotnine as p9
import warnings
warnings.filterwarnings('ignore')
!pip install nbconvert
```

```
zsh:1: /Users/yu/Documents/hs4002_ay26/hs4002_labs2026/.venv/bin/pip: bad
interpreter: /Users/yu/Documents/hs4002_ay26/hs4002labs_2026/.venv/bin/python3:
no such file or directory
Requirement already satisfied: nbconvert in /opt/anaconda3/lib/python3.12/site-
packages (7.16.4)
Requirement already satisfied: beautifulsoup4 in
/opt/anaconda3/lib/python3.12/site-packages (from nbconvert) (4.12.3)
Requirement already satisfied: bleach!=5.0.0 in
/opt/anaconda3/lib/python3.12/site-packages (from nbconvert) (4.1.0)
Requirement already satisfied: defusedxml in /opt/anaconda3/lib/python3.12/site-
packages (from nbconvert) (0.7.1)
Requirement already satisfied: jinja2>=3.0 in
/opt/anaconda3/lib/python3.12/site-packages (from nbconvert) (3.1.4)
Requirement already satisfied: jupyter-core>=4.7 in
/opt/anaconda3/lib/python3.12/site-packages (from nbconvert) (5.7.2)
Requirement already satisfied: jupyterlab-pygments in
/opt/anaconda3/lib/python3.12/site-packages (from nbconvert) (0.1.2)
Requirement already satisfied: markupsafe>=2.0 in
/opt/anaconda3/lib/python3.12/site-packages (from nbconvert) (2.1.3)
Requirement already satisfied: mistune<4,>=2.0.3 in
/opt/anaconda3/lib/python3.12/site-packages (from nbconvert) (2.0.4)
Requirement already satisfied: nbclient>=0.5.0 in
/opt/anaconda3/lib/python3.12/site-packages (from nbconvert) (0.8.0)
```

```
Requirement already satisfied: nbformat>=5.7 in
/opt/anaconda3/lib/python3.12/site-packages (from nbconvert) (5.10.4)
Requirement already satisfied: packaging in /opt/anaconda3/lib/python3.12/site-
packages (from nbconvert) (24.1)
Requirement already satisfied: pandocfilters>=1.4.1 in
/opt/anaconda3/lib/python3.12/site-packages (from nbconvert) (1.5.0)
Requirement already satisfied: pygments>=2.4.1 in
/opt/anaconda3/lib/python3.12/site-packages (from nbconvert) (2.15.1)
Requirement already satisfied: tinycss2 in /opt/anaconda3/lib/python3.12/site-
packages (from nbconvert) (1.2.1)
Requirement already satisfied: traitlets>=5.1 in
/opt/anaconda3/lib/python3.12/site-packages (from nbconvert) (5.14.3)
Requirement already satisfied: six>=1.9.0 in /opt/anaconda3/lib/python3.12/site-
packages (from bleach!=5.0.0->nbconvert) (1.16.0)
Requirement already satisfied: webencodings in
/opt/anaconda3/lib/python3.12/site-packages (from bleach!=5.0.0->nbconvert)
(0.5.1)
Requirement already satisfied: platformdirs>=2.5 in
/opt/anaconda3/lib/python3.12/site-packages (from jupyter-core>=4.7->nbconvert)
(3.10.0)
Requirement already satisfied: jupyter-client>=6.1.12 in
/opt/anaconda3/lib/python3.12/site-packages (from nbclient>=0.5.0->nbconvert)
(8.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
/opt/anaconda3/lib/python3.12/site-packages (from jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert) (2.9.0.post0)
Requirement already satisfied: pyzmq>=23.0 in
/opt/anaconda3/lib/python3.12/site-packages (from jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert) (25.1.2)
Requirement already satisfied: tornado>=6.2 in
/opt/anaconda3/lib/python3.12/site-packages (from jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert) (6.4.1)
Requirement already satisfied: fastjsonschema>=2.15 in
/opt/anaconda3/lib/python3.12/site-packages (from nbformat>=5.7->nbconvert)
(2.16.2)
Requirement already satisfied: jsonschema>=2.6 in
/opt/anaconda3/lib/python3.12/site-packages (from nbformat>=5.7->nbconvert)
(4.23.0)
Requirement already satisfied: attrs>=22.2.0 in
/opt/anaconda3/lib/python3.12/site-packages (from
jsonschema>=2.6->nbformat>=5.7->nbconvert) (23.1.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
/opt/anaconda3/lib/python3.12/site-packages (from
jsonschema>=2.6->nbformat>=5.7->nbconvert) (2023.7.1)
Requirement already satisfied: referencing>=0.28.4 in
/opt/anaconda3/lib/python3.12/site-packages (from
jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.30.2)
Requirement already satisfied: rpds-py>=0.7.1 in
```

```
/opt/anaconda3/lib/python3.12/site-packages (from
jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.10.6)
Requirement already satisfied: soupsieve>1.2 in
/opt/anaconda3/lib/python3.12/site-packages (from beautifulsoup4->nbconvert)
(2.5)
```

2 Python Fundamentals

Before working with real survey data, it's worth getting comfortable with the building blocks every Python program is made of: variables, strings, lists, dictionaries, conditionals, loops, and functions. Each of these will show up again, in a pandas-flavoured form, later in this notebook.

2.1 Variables and Data Types

Create three variables: your name (a `str`), your age (an `int`), and your height in meters (a `float`). Print each one alongside its data type.

Hint: `type(some_variable)` tells you what data type something is.

[40]:
```
name = "Jamie" # str is just letters. need to put it in ''
age = 20 # int are just integer numbers
height = 1.68 # floats are numbers with decimal points

print(name, type(name))
```

```
Jamie <class 'str'>
```

2.2 Conditionals (if / elif / else)

Write a conditional that decides whether a respondent counts as a "minor", "young adult", or "adult" based on their age.

Hint: Python uses `if`, `elif`, `else` — and indentation (not curly braces) marks what belongs inside each branch.

[41]:
```
respondent_age = 17

if respondent_age < 21:
    category = "minor"
elif respondent_age < 30:
    category = "young adult"
else:
    category = "adult"

print(category)
```

```
minor
```

2.3 Functions

Wrap the minor / young adult / adult logic from earlier into a reusable **function**, then apply it to every age in `ages`.

Hint: `def function_name(parameter):` starts a function; `return` sends a value back to whoever called it.

[42]:
```
def categorize_age(age_input):
    if age_input < 18:
        return "minor"
    elif age_input  < 30:
        return "young adult"
    else:
        return "adult"

categorize_age(30)
```

[42]:
```
'adult'
```

2.4 Objects, Attributes, and Methods

Everything in Python is an **object** — including the DataFrames you're about to work with. An object bundles together data (**attributes**, accessed with `object.attribute`, no parentheses) and behavior (**methods**, accessed with `object.method()`, always with parentheses — even if empty).

You'll see this pattern constantly for the rest of this notebook: `df.shape` and `df.columns` are attributes; `df.dropna()` and `df.sample()` are methods.

[43]:
```
# attributes are facts about objects, accessed using a . no brackets!
# e.g. df.shape, df.columns

example_list = [3, 1, 2] # defines what the object example_list is

print(len(example_list))      # a function -- do something to an object. can␣
 ↪work on multiple objects
example_list.sort()           # a method -- where the object does something by␣
 ↪writing this code. has brackets
print(example_list)

example_str = "hello"
print(example_str.upper())    # another method, even on a plain string
```

```
3
[1, 2, 3]
HELLO
```

2.5 Reading the Data

Hint: use `pd.read_csv()`. The data object in pandas is called a **DataFrame**.

[44]:
```
df = pd.read_csv("gss2022_mini.csv")
df
```

[44]:

	year	id	wrkstat	occ10	prestg10	marital	educ	age	degree	\
0	2022	1	1.0	430.0	39.0	3.0	16.0	72.0	3	
1	2022	2	5.0	50.0	53.0	1.0	18.0	80.0	4	
2	2022	3	1.0	4610.0	48.0	3.0	12.0	57.0	1	
3	2022	4	3.0	4120.0	34.0	5.0	16.0	23.0	3	
4	2022	5	8.0	7330.0	38.0	5.0	14.0	62.0	1	
…	…	…	…	…	…	…	…	…	…	
3539	2022	3541	1.0	5300.0	30.0	5.0	12.0	22.0	1	
3540	2022	3542	1.0	2310.0	61.0	1.0	19.0	29.0	4	
3541	2022	3543	1.0	3220.0	62.0	5.0	15.0	32.0	2	
3542	2022	3544	1.0	2310.0	61.0	1.0	17.0	49.0	4	
3543	2022	3545	1.0	6120.0	42.0	1.0	20.0	50.0	4	

	major1	…	cvdmov	cvdcreat	cvdrdg	cvdtour	cvdstmus	cvdarmus	\
0	44.0	…	NaN	NaN	NaN	NaN	NaN	NaN	
1	9.0	…	NaN	NaN	NaN	NaN	NaN	NaN	
2	NaN	…	NaN	NaN	NaN	NaN	NaN	NaN	
3	44.0	…	NaN	NaN	NaN	NaN	NaN	NaN	
4	NaN	…	4.0	4.0	4.0	4.0	4.0	2.0	
…	…	…	…	…	…	…	…	…	
3539	NaN	…	NaN	NaN	NaN	NaN	NaN	NaN	
3540	45.0	…	NaN	NaN	NaN	NaN	NaN	NaN	
3541	55.0	…	NaN	NaN	NaN	NaN	NaN	NaN	
3542	26.0	…	NaN	NaN	NaN	NaN	NaN	NaN	
3543	8.0	…	NaN	NaN	NaN	NaN	NaN	NaN	

	cvdstpo	cvdarpo	cvdclass	cvdpod
0	NaN	NaN	NaN	NaN
1	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN
4	4.0	2.0	4.0	4.0
…	…	…	…	
3539	NaN	NaN	NaN	NaN
3540	NaN	NaN	NaN	NaN
3541	NaN	NaN	NaN	NaN
3542	NaN	NaN	NaN	NaN
3543	NaN	NaN	NaN	NaN

[3544 rows x 49 columns]

2.6 How many rows does the file contain?

Hint: `len(df)` or `df.shape[0]`.

[45]:
```
len(df) # asks for the number of rows
```

[45]:
```
3544
```

[46]:
```
df.shape # gives the rows and columns
```

[46]:
```
(3544, 49)
```

[47]:
```
df.shape[0] # gives only the first item (i.e. the no. of rows)
```

[47]:
```
3544
```

2.7 Print the names of all columns

[48]:
```
df.columns # prints column names
```

[48]:
```
Index(['year', 'id', 'wrkstat', 'occ10', 'prestg10', 'marital', 'educ', 'age',
       'degree', 'major1', 'biokids', 'sex', 'race', 'income16', 'rincom16',
       'pres16', 'pres20', 'happy', 'health', 'natspac', 'natenvir', 'natheal',
       'natcrime', 'reg16', 'yrlvmus', 'yrartxbt', 'yrmovie', 'artsout',
       'yrcreat', 'yrrdg', 'yrtour', 'yrstmus', 'yrarmus', 'yrstpo', 'yrarpo',
       'yrclass', 'yrpod', 'cvdlvmus', 'cvdart', 'cvdmov', 'cvdcreat',
       'cvdrdg', 'cvdtour', 'cvdstmus', 'cvdarmus', 'cvdstpo', 'cvdarpo',
       'cvdclass', 'cvdpod'],
      dtype='str')
```

3 Selecting and Sampling

Narrow the dataset down to five variables: survey ID (`id`), employment status (`wrkstat`), occupational prestige (`prestg10`), age (`age`), and gender (`sex`).

Hint: index the DataFrame with a list of column names.

[56]:
```
chosen_variables = ["id", "wrkstat", "prestg10", "age", "sex"]

df2 = df[chosen_variables].copy()
# df[chosen_variables] cuts out all the data except the 5 needed ones
# put .copy() to create a new table with this data
# use .copy () to avoid accidentally changing the original df
```

3.1 Checking for Missing Data First

Before dropping any rows, it's worth seeing *how much* is actually missing, and in which columns — dropping blindly can silently throw away more data than you realize.

Hint: `.isna()` marks each cell `True`/`False` for missing; `.sum()` on top of that counts how many `True`s per column.

[57]:
```
print(df2.isna().sum())
# .isna finds the missing values, .sum counts the number of missing values
```

```
id            0
wrkstat       7
prestg10    218
age         208
sex          20
dtype: int64
```

3.2 Dropping rows with missing values

Hint: use `.dropna()`.

[58]:
```
df2 = df2.dropna()
df2
```

[58]:

	id	wrkstat	prestg10	age	sex
0	1	1.0	39.0	72.0	2.0
1	2	5.0	53.0	80.0	1.0
2	3	1.0	48.0	57.0	2.0
3	4	3.0	34.0	23.0	2.0
4	5	8.0	38.0	62.0	1.0
…	…	…	…	…	…
3539	3541	1.0	30.0	22.0	2.0
3540	3542	1.0	61.0	29.0	2.0
3541	3543	1.0	62.0	32.0	1.0
3542	3544	1.0	61.0	49.0	2.0
3543	3545	1.0	42.0	50.0	1.0

[3145 rows x 5 columns]

3.3 Taking a sample of 500

Setting a random seed makes the sample reproducible.

Hint: use `.sample(n=..., random_state=...)`.

[59]:
```
df_mini = df2.sample(n=500, random_state=47)
# do this to extract 500 values
# random_state ensures that the same 500 values will be extracted when this␣
 ↪piece of code is run
```

3.4 Sanity check — should be 500 rows and 5 columns

[60]:
```
print(df_mini.shape)
```

```
(500, 5)
```

3.5 Saving Your Cleaned Data

Write `df_mini` out to a new file, so you (or someone else) can pick up from here without re-running everything above.

Hint: `.to_csv(..., index=False)`. Name the output differently from the input, so you don't overwrite your source data.

[61]:
```
df_mini.to_csv("gss2022_mini_cleaned.csv", index=False)
# index = false tells python not to save the row numbers
```

4 Making New Variables

Recode `wrkstat` into a working/not-working binary. Full-time (1), part-time (2), and with-a-job-but-not-at-work (3) are all counted as working.

Hint: `np.where(condition, if_true, if_false)` lets you recode based on a condition.

[62]:
```
df_mini = df_mini.assign(

working = np.where(df_mini['wrkstat'] <= 3, 'Working', 'Not Working'),
sex_woman = np.where(df_mini['sex'] == 2, 'Woman', 'Not Woman') # recode values␣
 ↪such that all 2 will be classified as women, whiel all other values are␣
 ↪turned into 'not women'
)
df_mini
```

[62]:

	id	wrkstat	prestg10	age	sex	working	sex_woman
218	219	5.0	41.0	44.0	1.0	Not Working	Not Woman
1653	1654	1.0	38.0	41.0	2.0	Working	Woman
3487	3489	5.0	64.0	64.0	1.0	Not Working	Not Woman
363	364	5.0	35.0	70.0	1.0	Not Working	Not Woman
3366	3367	1.0	66.0	35.0	1.0	Working	Not Woman
…	…	…	…	…	…	…	…
1457	1458	4.0	31.0	43.0	2.0	Not Working	Woman
821	822	1.0	38.0	65.0	1.0	Working	Not Woman
1910	1911	5.0	35.0	69.0	2.0	Not Working	Woman
1309	1310	5.0	61.0	72.0	2.0	Not Working	Woman
2621	2622	5.0	47.0	60.0	1.0	Not Working	Not Woman

[500 rows x 7 columns]

4.1 Sanity check — print the breakdown of each new variable

[63]:
```
print(df_mini['working'].value_counts())
print(df_mini['sex_woman'].value_counts())
```

```
working
Working        292
```

```
Not Working    208
Name: count, dtype: int64
sex_woman
Woman        258
Not Woman    242
Name: count, dtype: int64
```

4.2 Casting a Column's Type

Sometimes you need to change a column's *type* without changing its values — different from recoding, which is what `sex_woman` did above.

Hint: `.astype(str)` casts a column to text.

[64]:
```
df_mini['sex_str'] = df_mini['sex'].astype(str) # changes the type of x into␣
 ↪string variable
print(df_mini['sex_str'].dtype)
```

```
str
```

5 Producing a Table

We want the mean occupational prestige among working men and women.

5.1 Filter for working individuals

Hint: use boolean indexing.

[65]:
```
df_wrk = df_mini.loc[df_mini['working'] == 'Working',:]
# create a dataframe called df_wrk
# .loc means to access the rows and columns of the dataframe df_mini
# only include data with working under the working variable!
df_wrk
```

[65]:

	id	wrkstat	prestg10	age	sex	working	sex_woman	sex_str
1653	1654	1.0	38.0	41.0	2.0	Working	Woman	2.0
3366	3367	1.0	66.0	35.0	1.0	Working	Not Woman	1.0
1935	1936	1.0	65.0	35.0	1.0	Working	Not Woman	1.0
1696	1697	1.0	35.0	60.0	2.0	Working	Woman	2.0
461	462	1.0	64.0	56.0	2.0	Working	Woman	2.0
…	…	…	…	…	…	…	…	…
330	331	3.0	60.0	20.0	1.0	Working	Not Woman	1.0
1924	1925	1.0	47.0	32.0	1.0	Working	Not Woman	1.0
3203	3204	2.0	35.0	35.0	2.0	Working	Woman	2.0
739	740	1.0	61.0	43.0	2.0	Working	Woman	2.0
821	822	1.0	38.0	65.0	1.0	Working	Not Woman	1.0

[292 rows x 8 columns]

5.2 Filtering on More Than One Condition

Narrow `df_mini` down to just the working women.

Hint: combining conditions in pandas uses `&` (and) / `|` (or) — not Python's plain `and`/`or` — and each condition needs its own parentheses.

[66]:
```
df_working_women = df_mini[
    (df_mini['working'] == 'Working') & (df_mini['sex_woman'] == 'Woman')
]
print(df_working_women.shape)
```

```
(147, 8)
```

5.3 Mean occupational prestige by gender

We'll do it the long way — split into two dataframes first (useful for learning), then retrieve the mean of each.

[67]:
```
df_men   = df_wrk[df_wrk['sex_woman'] == 'Not Woman']
df_women = df_wrk[df_wrk['sex_woman'] == 'Woman']

print(df_men['prestg10'].mean())
print(df_women['prestg10'].mean())
```

```
45.86896551724138
45.74149659863946
```

5.4 Building a summary table

[68]:
```
first_table = pd.DataFrame({
    'gender':          ['Men', 'Women'],
    'mean_occ_prestg': [df_men['prestg10'].mean(), df_women['prestg10'].mean()]
})
# creates a table, where the headers are gender and mean occupational prestige
first_table
```

[68]:

	gender	mean_occ_prestg
0	Men	45.868966
1	Women	45.741497

5.5 Exporting Your Table for Use in Word

pandas can save your table as a CSV file. A CSV stores table data in a format that Excel and Google Sheets can easily open.

After running the next cell:

1. Find the new CSV file in the same folder as this notebook.
2. Open it in Excel or import it into Google Sheets.
3. Select and copy the table.

4. Paste the table into your Word document.

Opening the CSV directly in Word is not recommended because Word may show it as plain, comma-separated text.

Hint: `.to_csv(..., index=False)` prevents pandas from adding an unnecessary column of row numbers.

[69]:
```
first_table.to_csv(
    "mean_occupational_prestige_by_gender.csv",
    index=False, # tells python not to save row numbers!
    float_format="%.1f" # keep decimal numbers to 1 d.p.
)
```

[70]:
```
# Optional: export the table as LaTeX code for a LaTeX document
print(first_table.to_latex(
    index=False,
    float_format="%.1f",
    caption="Mean Occupational Prestige by Gender",
    header=["Gender", "Mean Occupational Prestige"]
))
```

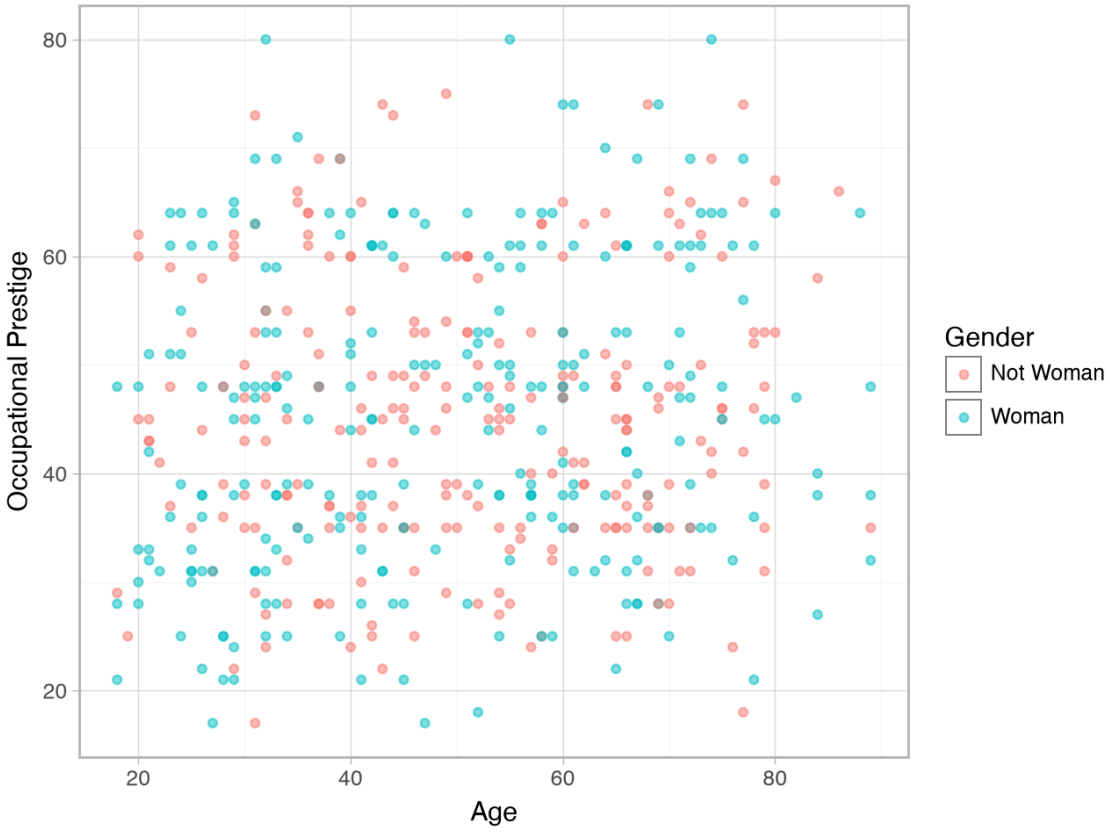
```
\begin{table}
\caption{Mean Occupational Prestige by Gender}
\begin{tabular}{lr}
\toprule
Gender & Mean Occupational Prestige \\
\midrule
Men & 45.9 \\
Women & 45.7 \\
\bottomrule
\end{tabular}
\end{table}
```

6 Making a Scatterplot

Plot the relationship between age and occupational prestige, coloured by gender.

[71]:
```
(
    p9.ggplot(df_mini) + # create a dot graph from the framework df_mini
    p9.geom_point(p9.aes(x='age', y='prestg10', color='sex_woman'), alpha=0.5)␣
 ↪+ # plot the dots. alpha controls intensity of colour of dots
    p9.theme_light() + # choose a light background theme
    p9.labs(x='Age', y='Occupational Prestige', color='Gender') # add labels
)
```

[71]:


Occupational Prestige
80
60
40
20
20
40
60
80
Age
Gender
Not Woman
Woman